DEVELOPING A MODEL-INFORMED DRUG DISCOVERY INTERFACE: A SOFTWARE ENGINEERING PROJECT

by

HANNAH HOUSAND
hjh21a

A PROJECT

Presented before Graduate Faculty of the

FLORIDA STATE UNIVERSITY DEPARTMENT OF COMPUTER SCIENCE

In Partial Fulfillment of the Requirements for the Degree

MASTER OF SCIENCE

In

COMPUTER SCIENCE

2026

Panel Members

Dr. Sonia Haiduc (Advisor)
Dr. Xiaonan Zhang
Dr. Christopher Mills

Contents

1 Introduction

Model-informed drug discovery (MIDD) is an increasingly important part of the pharmaceutical development process. By using computational models to predict how a drug will behave in the human body, researchers can make more informed decisions about whether a compound is worth progressing through animal testing and clinical trials. This reduces cost, time, and unnecessary risk to patients. Additionaly, it can be used to improve existing compounds and how to decide which of a set of candidate compounds ahs the best potential efficacy.

However, a significant gap exists in the current landscape of MIDD tooling. Most existing software is designed with modelers in mind — users who are comfortable writing and editing code. In practice, the people with the deepest knowledge of a drug's properties are often medicinal chemists, who may have little to no programming background. This creates a bottleneck: the domain experts who stand to benefit most from modeling are effectively locked out of the process, forced to rely on a modeler as an intermediary just to interact with a simulation.

This project addresses that gap. The goal was to develop a MIDD application, named Omni, accessible to non-modelers — particularly medicinal chemists, who represent the primary target audience — without sacrificing the depth of information available to them. However, the potential user base extends beyond medicinal chemists alone. Pharmacologists, toxicologists, biologists, formulation scientists, and biomarker researchers are among the groups who stand to benefit from accessible PBPK modeling tools, and Omni was designed with this broader audience in mind. The application allows users to manage projects and compounds, view and compare physiological data across species including humans, and run interactive simulations where parameters can be adjusted in real time and results update immediately. This last point addresses another shortcoming of existing tools, which are often too slow to support the kind of exploratory, iterative analysis that is most useful in early drug discovery.

This report makes the following contributions. First, a user interface designed specifically for non-modelers, lowering the barrier to entry for medicinal chemists who need to interact with PBPK models without writing code. Second, a real-time interactive simulation environment where users can adjust physiological and compound parameters and observe the effect on model outputs immediately, addressing a performance gap present in existing tools. Third, a custom pivot grid component built on top of Avalonia's tree grid, designed to display the complex hierarchical physiological data that underlies PBPK models in a familiar spreadsheet-like format. Finally, a structured Minimum Viable Product (MVP) scoping process grounded in real stakeholder research, involving domain experts including a PBPK modeler and a pharmacologist.

The remainder of this report is structured as follows. Section 2 provides background on MIDD and reviews existing tools and related literature. Section 3 covers requirements engineering, including stakeholder identification, the interview process, and the evolution of requirements toward an MVP. Section 4 describes the system design, including architecture, data design, and UI decisions. Section 5 details the implementation, covering the technology stack, key features, and engineering challenges encountered. Section 6 presents the evaluation approach and results. Section 7 concludes with a summary and directions for future work.

2 Background and Related Work

2.1 Background

Physiologically-based pharmacokinetic (PBPK) modeling is a technique used in drug development to predict how a compound will behave in the bodies of humans and animals. Rather than relying solely on observed data, PBPK models integrate drug-specific properties with physiological data — such as organ volumes, blood flow rates, and tissue composition — to simulate how a drug is absorbed, distributed, metabolized, and excreted. Importantly, PBPK models can be parameterized using data from in vitro

experiments alone, meaning they can be applied before a compound has ever been tested in a living organism. This makes PBPK modeling a valuable tool in the drug development pipeline, where it is used to optimize dosing for special populations such as pediatric or renally impaired patients and reduce the need for animal testing. Predicting drug behavior at this early stage — before human or animal trials begin — is the central use case that Omni is designed to support.

2.2 Existing Tools

Several software tools currently exist to support PBPK modeling, each with notable limitations that motivated the development of this project.

GastroPlus, developed by Simulations Plus, is one of the most widely used commercial PBPK platforms in the pharmaceutical industry. It is a powerful tool capable of whole-body PBPK modeling, but its commercial licensing makes it prohibitively expensive for many research groups. Beyond cost, its complexity and steep learning curve make it poorly suited for users without a strong modeling background.

PK-Sim, part of the Open Systems Pharmacology Suite, offers an open-source alternative. While its accessibility is an advantage, the interface is dated and unintuitive, presenting a usability barrier for non-modelers. Additionally, PK-Sim is primarily focused on pharmacokinetics and offers limited support for pharmacodynamic modeling. Tools such as Tellurium are better suited for pharmacodynamics but are similarly inaccessible to users without technical backgrounds.

Together, these tools highlight a consistent gap in the existing landscape: current PBPK software is designed for modelers, not for the medicinal chemists and domain experts who are often best positioned to interpret and act on simulation results. This complexity is not solely a matter of requiring users to write code — tools like GastroPlus and PK-Sim do not require code by default. Rather, the barrier lies in the sheer number of configuration options these tools expose, and in a user interface that assumes familiarity with the underlying model structure. For a medicinal chemist without a modeling background, navigating these interfaces is intimidating regardless of whether any code is involved. Omni addresses this in two ways: by avoiding code creation entirely, and by deliberately minimizing and simplifying the information the UI asks for — favoring familiar paradigms like spreadsheets, sliders, and tags, and limiting prompts to concepts a medicinal chemist would already understand. This project aims to bridge that gap.

3 Requirements Engineering

3.1 Stakeholder Identification

Identifying the right stakeholders was an important first step in shaping the requirements for this project. Rather than designing in isolation, the goal was to gather perspectives from people who represent the range of users this tool is intended to serve, as well as those with deep technical knowledge of the domain.

Two primary stakeholders were consulted during the requirements gathering phase. The first was a PBPK modeler, who represents the more technical end of the intended user base. While this application is primarily designed for non-modelers, power users such as PBPK modelers stand to benefit from the tool as well. This stakeholder was also consulted on specific backend modeling questions that arose during development, making them valuable both as a domain expert and as a technical resource. The second stakeholder was a pharmacologist with prior experience contributing to the development of another modeling tool. This gave them a unique perspective — both as a representative of the target user group and as someone familiar with the challenges of building software for this domain. Both stakeholders are industry professionals and former colleagues of my partner, which facilitated access and ensured their feedback was grounded in real-world MIDD workflows. In addition to these formal stakeholder consultations, the design of Omni was informed by broader industry input gathered prior to the start of

this project. My partner's experience working with approximately 15 to 20 professionals at a pharmaceutical company provided indirect but valuable insight into the needs of the target user base. While these individuals had no direct exposure to Omni, the feedback and perspectives they shared informed many of the design and prioritization decisions made throughout development. This prior industry context strengthened the requirements gathering process and grounded Omni's design in real-world user needs.

In addition to these two stakeholders, a group of faculty and students at Florida State University have been identified as participants for upcoming usability evaluation sessions. These individuals were identified by reviewing published work from the FSU medicine department and selecting those whose research involved PBPK modeling or related areas. While their feedback has not yet been collected, they represent the broader population of potential users and will inform future iterations of the application.

3.2 Interview Process

Interviews with both stakeholders were conducted remotely via Zoom and arranged through email. Rather than following a rigid script, each session was structured as an open-ended conversation guided by a set of broad questions about the concept, design, and direction of Omni. This format was chosen deliberately given the exploratory nature of the project at this stage — the goal was to surface genuine reactions and domain insight rather than validate a fixed set of assumptions.

The session with the PBPK modeler was the more technically focused of the two. In addition to general feedback on the concept, it addressed specific backend questions, including discussion of the lumping procedure used for slowly and rapidly perfusing compartments, which informed decisions about how the simulation layer should handle tissue groupings. Beyond the technical discussion, the modeler responded positively to the overall concept, noting that he had not seen a tool quite like Omni before.

The session with the pharmacologist had a broader focus. While he expressed some skepticism about the challenge of gaining market entry as a new tool in an established space, his overall reaction to the concept was positive. He was particularly enthusiastic about the real-time interactive simulation feature, which aligns with one of Omni's core differentiators from existing tools.

3.3 Functional Requirements

The following functional requirements were derived from stakeholder interviews and iterative discussions throughout the project. They are organized by feature area.

- Model Setup and Configuration
  - Users can configure model settings such as absorption type and related parameters
  - Users can save a project and reload it in a future session without reentering data
  - Input fields include validation logic to prevent invalid model configurations
  - The UI should only prompt the user for data required for the particular setup selected
- Compound Management
  - Users can add compounds to a model and edit their associated properties
  - Compound data is stored and persisted as part of the project
- Simulation
  - Users can run a simulation and view the resulting outputs
  - Users can adjust parameters in real time while a simulation is running and observe immediate changes in results

  - Users can save simulation results and return to them in a later session
  - Users can compare simulation results against in vivo data
  - Users can view INCA and absorption plots
  - Users can edit plot display settings such as axis scaling and log transformation
- Physiological Data
  - Users can view physiological data underlying the model
  - Users can view this data in multiple formats
- Interface
  - The application supports both dark mode and light mode
  - Users can reorganize areas of the screen using a docking system similar to those found in professional desktop applications

3.4 Non-Functional Requirements

- Performance
  - Simulations must feel responsive and interactive, addressing a known limitation of existing PBPK tools
- Usability
  - The interface must be accessible to non-modelers, particularly medicinal chemists with little to no programming background
- Cross-Compatibility
  - The application must run on multiple operating system types without degradation in appearance or functionality
- Security
  - All data remains local to the user's machine. Given the sensitive nature of drug development data, the application does not transmit any information over the internet during normal operation
- Maintainability
  - The codebase must be loosely coupled and modular, allowing individual components to be understood, modified, and extended independently
  - The backend is structured with clear separation between models, supporting future extension toward a block-based modeling interface without requiring significant rearchitecting

3.5 Requirements Evolution and MVP Scoping

The original vision for Omni, internally referred to as Omni3, was significantly more ambitious than the application described in this report. The initial concept centered around a block-based modeling interface, inspired in part by visual programming environments like Scratch. Users would be able to drag blocks from a library into an assembly area to construct a model from scratch — for example, combining a whole-body PBPK block with a liver compartment and an oral absorption block to build a fully custom model. This approach was considered the most promising path toward making complex PBPK modeling accessible to non-modelers, offering both flexibility and a low barrier to entry.

However, several months into development it became clear that the block-based approach faced a fundamental technical blocker: the model sorting problem. Because models in Omni are structured as modular objects rather than a flat list of equations, the order in which they are executed during simulation must be determined at runtime. For a fixed, known model this order can be hardcoded. For a user-assembled model built from arbitrary combinations of blocks, no reliable general solution to this ordering problem has yet been identified. This remains an open problem.

Faced with this blocker, and recognizing that the codebase had diverged significantly from the tools and libraries settled on for the final product, the decision was made to largely restart development with a scoped MVP as the target. While this felt like a setback at the time, it was ultimately a deliberate and strategic choice. The MIDD space is evolving quickly, and there was a genuine concern that other tools might begin moving in a similar direction — simplifying modeling workflows for medicinal chemists. Getting a real, usable product in front of users as quickly as possible was therefore a priority, both to validate the concept and to establish Omni's presence in the space ahead of potential future commercialization.

The MVP retained the core features most likely to demonstrate value to early users: model setup and configuration, compound management, real-time interactive simulation, and physiological data viewing. The block-based interface remains the long-term vision and is discussed further in the future work section.

3.6 Product Backlog

Development of Omni was managed using a product backlog containing over 100 tasks spanning all feature areas of the application. Each task was assigned a priority score, a severity rating, a task type, and an owner — either Hannah (frontend), Conrad (backend/domain), or both. This allowed work to be organized and prioritized systematically across the project. The backlog was a living document, updated regularly as new requirements emerged from stakeholder conversations and self-evaluation. A representative sample of backlog items is shown in Table 1.

Task ID	Feature Area	Description	Priority	Assigned To	Status
32	Compounds	Finish building compound data columns	11	Hannah	Not Started
33	Compounds	Compound validation — unique names	15	Hannah	Not Started
60	PK	PK plot filtering — organ selection	14	Conrad	Not Started
62	Saved Results	Saved results display	23	Conrad	Not Started
88	Batch Simulation	Design batch simulation workflow	24	Both	Not Started
4	Infrastructure	Omni installers research	40	Hannah	In Progress

Table 1: Representative sample of Omni product backlog items

3.7 User Stories

User stories were used early in the project to capture the needs of different user types and to inform the functional requirements described in Section 3.3. The following stories represent the core use cases that shaped the design and feature set of Omni.

- As a modeler, I want to configure a PBPK model with specific physiological parameters so that I can simulate drug behavior accurately.
- As a pharmacologist, I want to add and test different compounds in a model so that I can compare their pharmacokinetic profiles.
- As a user, I want to run a simulation and adjust parameters in real time so that I can immediately see how changes affect model outputs.

- As a user, I want to view simulation outputs in multiple formats so that I can analyze results from different angles.
- As a user, I want to view and interact with physiological data in a structured grid so that I can understand the underlying model parameters.
- As a modeler, I want to save and reload model configurations so that I do not have to reenter parameters every session.
- As a pharmacologist, I want to compare simulation results against in vitro data so that I can validate model predictions against real observations.
- As a user, I want the application to run on my operating system of choice so that I am not restricted by platform compatibility.

4 System Design

4.1 System Overview

Omni is a cross-platform desktop application built to support model-informed drug discovery workflows. The application provides a graphical interface through which users can configure PBPK models, manage compounds, run simulations, and explore results — all without writing code. The frontend is written in C# using the Avalonia UI framework, and connects to a high-performance C++ simulation library that handles the underlying mathematical computation.

4.2 Architecture

Omni follows the Model-View-Presenter architectural pattern, chosen because it enforces a clean separation between the frontend and backend, reducing coupling between layers and making each component easier to understand and modify independently. The codebase is organized into four primary layers:

The Model layer contains the core data classes and business logic of the application. Each model represents a distinct concept within the domain — such as a compound, a physiological dataset, or a project configuration — and is implemented as a self-contained object. This separation was deliberate, as it supports the long-term goal of extending Omni toward a block-based modeling interface where individual models can be assembled dynamically.

The View layer contains all code relating to the visual presentation of the application. Views define what the user sees and interacts with, but contain no business logic themselves. The Presenter layer acts as the intermediary between Models and Views. It responds to user interactions, updates the Model layer accordingly, and determines how results are reflected in the View.

The Interop layer is a dedicated folder housing all code responsible for communication between the C# frontend and the C++ simulation library. Rather than calling the library directly from the Model or Presenter layers, this separation ensures that the complexity of bridging two languages is isolated in one place, keeping the rest of the codebase clean and maintainable.

Omni does not rely on a database. The primary data used by the application is physiological reference data, which is stored as CSV files bundled with the application. Project and compound data entered by the user is serialized to JSON and saved locally on the user's machine.

4.3 UI/UX Design

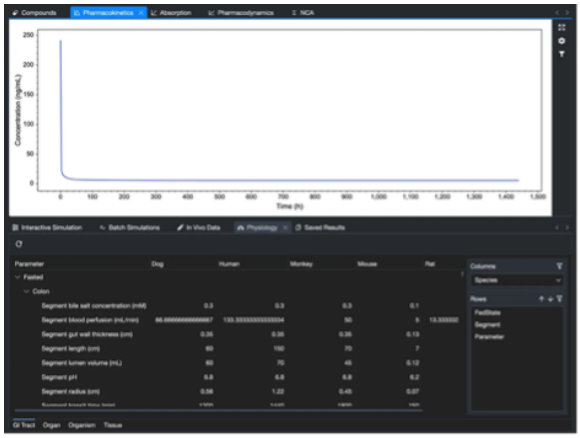

Figure 1: Physiological view of OMNI

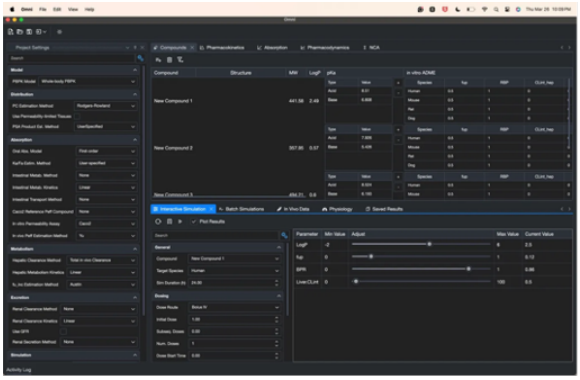

Figure 2: UI of OMNI

A core design goal of Omni was to create an interface that felt familiar and approachable to non-modelers, while still exposing the depth of functionality required by power users. Several key design decisions reflect this goal.

4.3.1 Layout and Docking

Omni uses a docking panel system that allows users to reorganize areas of the screen to suit their workflow, similar to professional desktop applications such as Visual Studio or RStudio. As shown in Figure 2, the interface is divided into distinct regions — a Project Settings panel on the left, a tabbed simulation and data area at the bottom, and a tabbed results and plot area at the top. Each region can be repositioned or resized, giving users control over their workspace. The application also supports both light and dark mode, with the dark theme shown in the screenshots above.

4.3.2 Project Settings Panel

The left panel provides access to all model configuration options, organized into collapsible sections including Model, Distribution, Absorption, Metabolism, Excretion, and Simulation. Each section exposes relevant parameters through dropdowns, checkboxes, and numeric inputs. This structure mirrors the mental model a pharmacokineticist would use when thinking about a drug's behavior, making it intuitive for domain experts to navigate.

4.3.3 Compounds View

The compounds tab presents compound data in a spreadsheet style grid, displaying properties such as molecular weight, LogP, and pKa alongside species specific in vitro ADME data. Users can add and manage multiple compounds, with each row representing a distinct compound and nested tables displaying associated property sets. This familiar tabular layout was chosen deliberately to lower the barrier to entry for users accustomed to working in tools like Excel.

4.3.4 Interactive Simulation

The interactive simulation panel allows users to run a simulation and adjust key compound parameters — such as LogP, fup, BPR, and Liver.CLint — using sliders in real time. Each slider displays its minimum value, maximum value, and current value, giving users immediate context for their adjustments. Results update instantly in the plot area above, allowing users to explore how parameter changes affect drug

behavior without rerunning the simulation manually. This addresses one of the most significant usability gaps identified in existing PBPK tools.

4.3.5 Physiology View and Custom Pivot Grid

The physiology tab displays the underlying physiological reference data used by the model in a hierarchical pivot grid, as shown in Figure 1. Data is organized by fed state, body segment, and parameter, with species shown as columns — allowing users to compare values across dog, human, monkey, mouse, and rat side by side. Users can customize the row grouping using the panel on the right, adjusting which dimensions are shown and in what order. The design and implementation of this custom pivot grid is discussed in detail in Section 5.

4.3.6 Splash Screen

Omni also includes a splash screen displayed on startup, contributing to a polished first impression and reinforcing the application's identity before the main interface loads.

5 Implementation

5.1 Technology Stack

Omni's frontend is written in C# using the .NET 9 framework, chosen primarily due to existing familiarity with the language, as well as its strong cross-platform support and the availability of high quality third-party component libraries. For the UI framework, Avalonia 11 was selected specifically for its ability to render a consistent, polished interface across multiple operating systems including Windows and macOS, directly supporting Omni's cross-compatibility requirement. The Fluent theme was applied to give the application a modern, professional appearance across both platforms.

!!! insert macOS and Windows screenshots The simulation backend is housed in a separate repository called OmniCore, which compiles to a C++ dynamic link library (DLL) that is loaded by the frontend at runtime. C++ was chosen for two reasons. First, it is among the fastest languages available, which was essential given the performance requirements identified during stakeholder interviews. Second, C++ is object-oriented, which allowed the backend to be structured as loosely coupled submodel classes rather than one large monolithic model. This modularity mirrors the frontend's Model-View-Presenter structure and lays the groundwork for the future block-based version of Omni. Keeping OmniCore as a separate repository also reinforced the separation between frontend and backend, making each easier to develop and maintain independently.

Several third-party libraries were used to avoid writing unnecessary code from scratch, in keeping with the project's guiding principle of reusing open source and available tooling where possible. A summary of the most significant dependencies is as follows:

- ActiPro for Avalonia provides the docking panel system that allows users to reorganize the interface layout, as well as application-wide styling and theming and a suite of polished basic input controls.
- ScottPlot was selected as the plotting library after evaluating several alternatives including LiveCharts. While LiveCharts was initially chosen for its flexibility and customization options, ScottPlot was ultimately adopted for its broader set of ready-to-use features out of the box and its better performance with large datasets. It handles all simulation result visualizations including pharmacokinetic, absorption, and pharmacodynamic plots.
- CommunityToolkit.Mvvm provides the ObservableProperty and ObservableValidator annotations used throughout the Model layer, significantly simplifying data binding and input validation logic

- Newtonsoft.Json handles JSON serialization for saving and loading project data
- TreeDataGrid.Avalonia provides the underlying tree grid component on which the custom pivot grid was built
- CsvHelper is used to read the physiological reference data from the CSV files bundled with the application
- AvaloniaEdit provides a code editor component used to display model code within the interface
- NCDK provides cheminformatics functionality used to render molecular structure diagrams from SMILES strings directly within the compounds view, allowing users to visually identify compounds at a glance

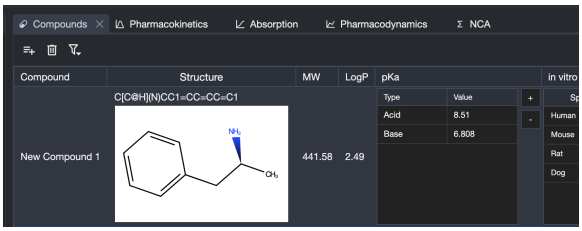

Figure 3: Molecular Diagram of Adderall based on SMILES

Project data is persisted using JSON serialization via Newtonsoft.Json, which was chosen for its simplicity and ease of implementation. While not the fastest serialization approach available, it was sufficient for the current MVP and allowed rapid development progress.

5.2 Custom Pivot Grid

One of the most significant implementation challenges in Omni was finding an effective way to display and interact with physiological reference data. This data is inherently complex — spanning multiple species, body segments, fed states, and parameter types — and users need to be able to compare values across these dimensions, filter out irrelevant data, and eventually edit values directly. For example, a researcher might want to compare colon blood perfusion rates across dog, human, and rat, or filter the view to show only human data as animal testing regulations evolve.

An initial attempt using a standard tree grid proved insufficient. While it could display the hierarchical structure of the data, it did not support the kind of flexible pivoting and filtering that would make the view genuinely useful. The solution was to build a custom pivot grid modeled closely on the familiar Excel pivot table interface, allowing users to choose which dimension appears as columns, reorder row groupings, and filter visible values — all through a configuration panel on the right side of the view.

The pivot grid is implemented as a hierarchical table abstraction built on top of Avalonia's TreeDataGrid component. At its lowest level, each displayed value is represented by a Cell object, which stores the column name, current value, default value, editability state, and an optional reference back to the originating raw data cell. This origin link allows edits made in the pivot grid to propagate back to the source data and supports visual change tracking through computed properties such as IsModified and Background.

Raw tabular data is stored in a CellTable, whose rows are composed of CellRowAdapter objects that expose cells through a column-based indexer. To support pivoted and grouped presentation, a second

table structure — HierarchicalCellTable — is introduced, whose rows are instances of HierarchicalCellRowAdapter. Unlike the flat CellRowAdapter, each hierarchical row contains both a dictionary of cells and an observable collection of child rows, enabling multi-level expansion in the UI. Each adapter subscribes to property changes from its underlying cells and raises notifications so that bindings in the grid refresh correctly when values change. An IsEmpty() method recursively checks whether a row or any of its descendants contain meaningful data, allowing empty branches to be suppressed from the final view.

The transformation from raw table data into the pivot grid is performed by PivotGridBuilder, which receives a PivotSettings object and constructs a HierarchicalCellTable in three stages. First, it determines the active pivot columns by reading the checked filter values associated with the selected pivot dimension. Second, it creates the visible grid columns, always placing Parameter first and appending one column per selected pivot value. Third, it generates the hierarchical row structure recursively based on the configured row hierarchy, creating parent rows with labels and empty value cells at non-leaf nodes, and populating leaf nodes by matching the current hierarchy context against the raw data.

Pivot configuration is managed through the PivotSettings class, which separates persisted user choices from values derived from the current dataset. Persisted state includes the selected pivot column, the row grouping order, and per-column checkbox filters. PivotSettings.InitializeAndMerge() rebuilds settings from the current schema while reapplying previously saved layout and filter choices where they remain valid, making the pivot grid resilient to reloads and schema changes while preserving as much of the user's prior configuration as possible.

5.3 Real-Time Interactive Simulation

The interactive simulation feature is one of Omni's core differentiators from existing PBPK tools. It allows users to adjust compound parameters using sliders and observe the effect on simulation outputs immediately, without manually rerunning the simulation. This section describes how that real-time workflow is implemented.

The feature is coordinated by InteractiveSimulationPresenter, which follows the Model-View-Presenter pattern and acts as the central layer between the user interface (InteractiveSimulationView), the global application state (Project.Instance), and the native C++ simulation engine accessed through OmniInterop. On initialization, the presenter assigns the simulation object as the data context for the parameter grid and dynamically populates the slider panel based on the current set of interactive simulation parameters. Each parameter is represented as a SliderItem with configurable minimum, maximum, and current values. To prevent excessive simulation requests when a user drags a slider slowly, the slider implementation includes built-in debounce logic, ensuring that simulation runs are only triggered once the slider value has settled rather than on every incremental change.

User interactions are handled through an event-driven architecture. The presenter subscribes to UI events including button clicks, checkbox changes, and slider adjustments. When a slider value changes, the presenter updates the corresponding field in the CompoundSettings object and immediately triggers a new simulation run. Changes to the selected compound or other simulation properties similarly invoke re-execution, ensuring that displayed results always reflect the current input configuration.

Simulation execution is encapsulated within the RunSimulation() method. This method creates simulator and model instances through the interop layer, constructs a SimulationSettings structure from the current application state, and passes both simulation and compound parameters to the native C++ engine. After execution, results are returned as a dictionary of time-series data and stored in Project.Instance.Simulation.Results, which automatically updates any bound UI components including the plot. Simulator and model handles are explicitly deleted after each run to prevent memory leaks across the repeated executions that real-time interaction produces.

The presenter also listens for changes in collections such as the compound list and interactive parameter

set. When these change — for example when a user adds a new compound — the slider panel updates dynamically to reflect the new configuration. This design enables a highly interactive workflow where users can iteratively explore the parameter space and observe results in real time.

5.4 C++/C# Interoperability

Communication between the C# frontend and the C++ simulation engine is handled through a dedicated interop layer implemented using Platform Invocation (P/Invoke). This layer exposes functions from the native library (OmnisimInterop) to the managed C# application, enabling the creation, execution, and destruction of simulation instances from within the frontend without exposing the complexity of the native code to the rest of the application.

Data exchange between the two languages is handled through explicitly defined structures — primarily SimulationSettings and CompoundSettings — which use sequential memory layouts to ensure compatibility between managed C# and unmanaged C++ memory models. Simulation results are returned from the native engine as pointers to NamedVector structures in native memory, which are then manually marshaled into managed C# collections. This process involves converting raw pointers into arrays and strings and copying the data into safe managed representations before it is passed to the rest of the application.

Memory management is handled explicitly throughout this process. After data retrieval, native cleanup functions are invoked to free memory allocated by the C++ engine, preventing leaks that would otherwise accumulate across the repeated simulation runs that real-time interaction produces. This design allows computationally intensive simulation work to execute at native C++ speed while keeping the frontend responsive and interactive.

5.5 AI-Assisted Development

Artificial intelligence tools were used selectively throughout the development of Omni to assist with particularly challenging implementation problems. Rather than generating large blocks of code, AI assistance was used to produce targeted snippets in response to specific technical blockers. This approach kept the codebase intentionally authored and understood by the development team rather than blindly adopted from generated output.

All AI-generated code was thoroughly reviewed line by line and validated through testing before being integrated into the project. This ensured that any suggested code was not only functional but also consistent with the existing architecture and efficient enough for Omni's performance requirements. Where generated snippets were inefficient or did not fit the codebase, they were modified or discarded.

5.6 Challenges and Limitations

5.6.1 Packaging

One of the most persistent unsolved challenges in Omni's development has been producing a proper installer that non-technical users can run on their own machines. The core difficulty lies in bundling two external dependencies alongside the application executable: the C++ simulation library (DLL) and the physiological reference data files (CSVs). Both must be present in the correct locations relative to the executable at runtime — the DLL for the simulation engine to load, and the CSV files for the physiological data to be accessible.

Several packaging approaches were investigated, including Avalonia's built-in publish tooling and third-party packaging tools, but none successfully handled the combination of a native DLL and external data files without breaking the application's ability to locate them at runtime. One potential solution involves ensuring these files are placed in a known, consistent location on the user's machine during installation, but this has not yet been fully implemented.

In the interim, distribution to trusted testers has been handled by zipping up the compiled project directory and sharing it directly. While this is not a viable long-term distribution strategy, it has been sufficient to support early testing across platforms. Producing a proper installer remains a priority for future development.

5.6.2 Model Sorting

A more fundamental unsolved challenge is the model sorting problem, which originates in the mathematical structure of PBPK simulations. Omni's backend is composed of individual submodels, each with defined inputs and outputs that may depend on the outputs of other submodels. For example, the output of one submodel may serve as an input to another, meaning the order in which submodels are executed during a simulation run directly affects the correctness of the results. For a fixed, known model such as whole-body PBPK, this execution order can be determined and hardcoded manually. However, for a dynamically assembled model — as would be required by the block-based Omni3 interface — the correct ordering must be determined automatically at runtime.

Several approaches were investigated to solve this problem, including separating each submodel's logic into an update function and a derivative function, allowing derivatives to be calculated across all submodels before updating state. However, this approach proved insufficient because some submodels require their update function to be called multiple times within a single simulation step, making a clean separation unreliable.

AI assistance was also consulted on this problem given its algorithmic nature, but no robust general solution was identified. For the current MVP, the problem is managed manually — my partner ensures that submodels within known model configurations are ordered correctly. However, this approach does not generalize to user-assembled models and remains the primary technical blocker preventing development of Omni3. Resolving the model sorting problem is therefore a critical area of future work. !!! insert figure showing submodel input/output dependencies

5.6.3 JSON Serialization Performance

Project and compound data in Omni is currently persisted using JSON serialization via Newtonsoft.Json. While this approach was straightforward to implement and integrate well with the CommunityToolkit.Mvvm ObservableProperty pattern used throughout the Model layer, JSON serialization is not the most performant option available. For the scale of data involved in the current MVP this has not presented a noticeable issue, but it may become a limitation as the application grows in complexity and the volume of persisted data increases.

More performant alternatives such as binary serialization exist and could be explored in future iterations. For now, the simplicity and developer-friendliness of JSON serialization was considered the right tradeoff for an MVP where development speed and maintainability were prioritized over raw performance.

5.7 Omni Web Application

In parallel with the desktop application, a web-based version of Omni was developed and hosted on !!! insert server/hosting details. The web application was built primarily to generate early user interest and facilitate feedback collection ahead of a wider release of the desktop application. Rather than replicating the full feature set of the desktop version, the web application focuses on showcasing Omni's most compelling feature — the real-time interactive simulation — with a small set of hardcoded models that users can explore immediately without any installation required. Beyond marketing, the web application also serves as a testing ground for candidate baseline models being considered for inclusion in the full desktop application. By exposing these models to real users in a lightweight environment, feedback can be gathered and models can be refined before they are integrated into Omni with its full feature set.

As shown in Figure X, the web interface presents a model configuration panel on the left alongside a plasma concentration-time plot and parameter sliders, allowing users to adjust dosing, physicochemical, and distribution parameters and observe simulation updates in real time. The interface includes tabs for plasma PK, organ concentrations, GI transit and absorption, NCA metrics, physiology data, and model source code.

Given that the web application is intended as a temporary marketing and feedback tool rather than a production product, much of its implementation was generated with AI assistance and adapted as needed.

The web application serves a dual purpose: it gives potential users a low-friction way to experience Omni's core value proposition, and it opens a channel for early conversations with the MIDD community that can inform future development of the desktop application.

6 Evaluation

6.1 Evaluation Goals

The evaluation of Omni pursues two parallel tracks. The first is a marketing and outreach effort aimed at building early awareness of the tool, gathering initial reactions from the broader MIDD community, and identifying potential beta testers ahead of a wider release. The second is a formal usability evaluation with domain-relevant participants at Florida State University, focused on assessing both the usability of the current interface and reactions to the block-based modeling concept that represents Omni's long-term vision.

6.2 Advertising and Outreach

A content plan has been developed to showcase Omni's key features across professional channels. Planned posts are designed to highlight the aspects of Omni most likely to resonate with the target audience, including the block-based Omni3 interface, the custom pivot grid for physiological data exploration, and the real-time interactive simulation where users can move sliders and observe immediate changes in model outputs.

Distribution is planned primarily through LinkedIn, targeting professionals in the pharmaceutical and drug development space. Additionally, outreach is planned to relevant mailing lists connected to the parent company's ODE solver community, where potential users with existing interest in computational modeling are likely to be found. While no posts have been published at the time of writing, this outreach effort represents an important step toward validating market interest in Omni and building a user base for future testing and development.

6.3 Planned User Evaluation

A formal usability evaluation is planned with faculty and students at Florida State University whose research involves PBPK modeling or related domains. Participants were identified by reviewing published work from the FSU medicine department. Initial contact has been made and interest has been confirmed, with evaluation sessions to be scheduled following the completion of this report.

The evaluation will pursue two goals. The first is to assess the usability of Omni's current MVP interface with users who represent the target audience — domain experts who are not necessarily experienced modelers. Participants will be given realistic use cases to work through, such as configuring a model with a known public compound, running a simulation, and interpreting the results. Observations will focus on where users encounter friction, what they find intuitive, and whether the interface achieves its goal of being accessible to non-modelers.

The second goal is to gather reactions to the block-based Omni3 concept, which will be presented to participants through mockups or the existing prototype. This feedback will inform prioritization of future development and help validate whether the block-based approach resonates with the intended user base.

Results and discussion from these sessions will be incorporated into this report following their completion.

7 Conclusion

7.1 Summary

This report has described the design and development of Omni, a cross-platform model-informed drug discovery application built to make PBPK modeling accessible to non-modelers. The project addressed a genuine gap in the existing landscape of MIDD tooling, where powerful software has historically been gatekept by its complexity and the assumption that users are comfortable writing and modifying code. Omni challenges that assumption by putting interactive, real-time simulation capabilities in the hands of medicinal chemists and pharmacologists who may have no programming background at all.

Beyond the software itself, this project represents a thorough demonstration of the agile software development process in practice. Requirements were gathered from real domain experts, translated into a product backlog, built into working software, and then revisited and revised as new constraints and insights emerged — most notably in the deliberate pivot from the ambitious Omni3 vision to a focused, testable MVP. While the full development lifecycle including long-term maintenance and large scale user evaluation remains ongoing, the project captures a complete and meaningful iteration of the agile cycle from requirements through implementation.

Several features of Omni are worth highlighting as particular contributions. The real-time interactive simulation, where users can adjust parameters with sliders and observe immediate changes in model outputs, directly addresses one of the most significant usability failures of existing PBPK tools. The custom pivot grid makes complex hierarchical physiological data navigable and familiar to users accustomed to spreadsheet environments. The molecular structure viewer allows users to visually identify compounds at a glance. And the display of underlying model source code within the interface has a practical implication beyond usability — regulatory submissions to the FDA often require model code to be provided, and Omni makes that code readily accessible to users who would otherwise never see it.

At its core, Omni is about accessibility. The people who most need to interact with PBPK models are often the same people who find existing tools the most intimidating. Omni aims to close that gap — not by simplifying the science, but by removing the technical barriers that prevent domain experts from engaging with it.

7.2 Future Work

The most significant direction for future work is the development of Omni3, the block-based modeling interface that represented the original vision for this project. As described in Section 3.5, the block-based approach would allow users to assemble custom PBPK models by dragging and connecting modular components — making model construction as intuitive as the simulation experience Omni already provides. Realizing this vision depends primarily on resolving the model sorting problem described in Section 5.6, which remains the key technical blocker.

Additional areas for future development include producing a proper installer to replace the current zip-based distribution method, expanding the set of supported model types, incorporating results from the planned FSU user evaluation sessions, and continuing the advertising and outreach effort to build a

broader user base. The web application will also continue to serve as a channel for early feedback and community engagement as Omni grows toward a wider release.

!!!need to add more references

References

[1] Avalonia UI. *Avalonia Documentation*. Available at: https://docs.avaloniaui.net

[2] Microsoft. *CommunityToolkit.Mvvm Documentation*. Available at: https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/

[3] Actipro Software. *ActiPro for Avalonia Documentation*. Available at: https://www.actiprosoftware.com

[4] ScottPlot. *ScottPlot Documentation*. Available at: https://scottplot.net